

Appendix

A Training Transformation Weights

To maximize the accuracy of trained weights when deployed in real-world scenarios, we simulate the reuse patterns of runtime systems for each sender-receiver model pair during training. For instance, in multi-turn conversation scenarios where the roles of the user and the assistant propagate alternately, context from previous turns is shared. When switching to a different model (e.g., via a router), the new model only needs to append the latest turn rather than processing the entire context history. We simulate this context reuse pattern by splitting each conversation into three segments: *context messages* (shared past context), *append messages* (newly added content), and *answer messages* (ground truth for decoding). During the training pipeline, the sender model first prefills the context messages to generate the attention map. Subsequently, the receiver model reuses this attention map while prefilling the context. Finally, we concatenate and feed the append and answer messages to receiver while masking the non-answer tokens so that the loss is computed solely on the answers.

However, due to the use of fused attention kernel libraries [11, 78], the attention map is not explicitly instantiated in GPU memory. Consequently, we must reconstruct the sender's attention map for the receiver to reuse. A significant challenge is that reconstructing the full attention map consumes quadratic memory relative to the sequence length, leading to frequent Out-of-Memory (OOM). To mitigate this, we leverage FlashAttention kernels during the receiver's reuse stage. Let sender query $Q = [q_1, \dots, q_m]$, sender key $K = [k_1, \dots, k_m]$ (expanded to match query heads if using Grouped-Query Attention [2]), transform weights $W \in \mathbb{R}^{m \times n}$, and receiver value $V = [v_1, \dots, v_n]$. Denoting the attention scores as $X_i = \text{softmax}(\frac{q_i k_i^T}{\text{scale}})$, for each output attention head j , the output can be formulated as:

$$O_j = \sum_{i=1}^m W_{ij} (X_i V_j)$$

We can utilize $\text{flash_attn}(q_i, k_i, v_j) = X_i V_j$ to avoid explicitly computing the attention matrix X_i , thereby significantly saving memory. More details can be found in Algorithm 3.

Algorithm 3 Memory-Efficient Attention Map Reuse in Training

Require: Sender $Q, K \in \mathbb{R}^{B \times L \times H_{in} \times D}$, receiver $V \in \mathbb{R}^{B \times L \times H_{out} \times D}$, Weights $W \in \mathbb{R}^{H_{in} \times H_{out}}$

- 1: Initialize output tensor $O \in \mathbb{R}^{B \times L \times H_{out} \times D}$
 - 2: $\tilde{W} \leftarrow \text{softmax}(W, \text{dim} = 0)$
 - 3: **for** $j = 1$ to H_{out} **do**
 - 4: Extract $V_j \leftarrow V[:, :, j, :]$ with shape $B \times L \times D$
 - 5: Broadcast V_j to H_{in} and multiply by $\{\tilde{W}_{i,j}\}_{i=1}^{H_{in}}$ to get V'_{input}
 - 6: $O_{\text{temp}} \leftarrow \text{FlashAttn}(Q, K, V'_{\text{input}})$
 - 7: $O[:, :, j, :] \leftarrow \sum_{i=1}^{H_{in}} O_{\text{temp}}[:, :, i, :]$
 - 8: **end for**
 - 9: **return** O
-

B Experiments

B.1 Datasets

We evaluate MAPFLOW on three diverse datasets, with their statistics provided in Table 6.

Table 6. Summary of evaluation datasets, task types, and context lengths.

Dataset	Type	Context Length
HotpotQA [74]	Question Answering	4–8k
RULER [25]	Question Answering	20k
LMSYS [86]	Conversation	12–20k
ShareGPT4 [10]	Conversation	12–20k
Multi-News [17]	Summarization	8–12k

Table 7. Popular model families and their major variants. Within each family, all variants share the same tokenizer.

Tokenizer	Popular Model Variants
Llama-3 (128k)	Llama 3, Llama 3.1, Llama 3.2, Llama 3.3
Qwen-2.5 (151k)	Qwen2, Qwen2.5, Qwen2.5-Coder, Qwen2.5-Math
Gemma (256k)	Gemma, Gemma1.1, Gemma2, CodeGemma, RecurrentGemma, ShieldGemma
Mistral (131k)	Mistral Nemo, Pixtral, Mistral Small, Ministral, Devstral, Magistral-Small

Table 8. Empirical sparsity ratio k/L across different sequence intervals under varying top- p thresholds.

top- p	0–4k	4–8k	8–12k	12–16k	16–20k	20–24k
0.7	0.1939	0.0978	0.0899	0.0764	0.0531	0.0453
0.8	0.2836	0.1642	0.1554	0.1338	0.0981	0.0802
0.9	0.4893	0.3368	0.3213	0.2802	0.2296	0.1865

B.2 Heatmaps on More Model Pairs

Figure 12 further provides the TV-distance heatmaps on HotpotQA, including Qwen2.5 14B→7B, Ministral3 14B→8B, and Llama3.1 70B→8B, which is deferred to the Appendix due to space constraints. The results show similar layer mapping patterns on these model combinations.

B.3 Sparsity Ratio Analysis

We analyze the effective sparsity ratio induced by block-level top- p filtering across different sequence intervals. Specifically, we measure the average number of preserved key positions per query token, denoted as k , under varying context lengths. The following experiments are conducted using the Qwen3-8B to Qwen3-1.7B model pair on the ShareGPT4 dataset. The empirical sparsity ratio is calculated by k/L . Table 8 reports the measured values of k/L , showing that the sparsity ratio decreases as the sequence length increases. For example, under $p = 0.7$, k/L decreases from 0.19 in the 0–4k interval to 0.045 in the 20–24k interval. This trend explained the observed plateau in memory usage, as the number of retained key positions grows much slower than the sequence length. These observations also empirically justify the complexity reduction discussed in section 4 where $k \ll L$ holds in practice.

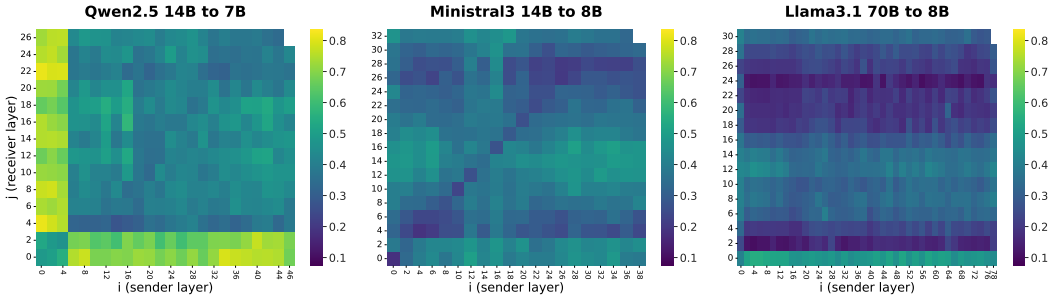


Fig. 12. TV-distance heatmaps for linear fits between every sender–receiver layer pair, for three additional model pairs: Qwen2.5 14B→7B, Ministral3 14B→8B, and Llama3.1 70B→8B. The x-axis is the sender-layer index, and the y-axis is the receiver-layer index. Entry (i, j) is the TV distance between the attention map fitted from sender layer i and the original attention map of receiver layer j ; lower values indicate a better fit.

C Kernel Design Details

C.1 FlexPrefill Configuration for Hybrid Reuse

For the 20K RULER hybrid experiment in Section 7.3, we use the FlexPrefill settings $\gamma = 0.1$ and $\tau = 0.1$, with a minimum attention budget of 1024 tokens. The hybrid controller measures the average retained key density k/L after MapFlow’s top- p compression. It dispatches dense maps (above the pair-specific 5% or 8% threshold) to FlexPrefill and uses MapFlow’s sparse fused retrieval for the remaining maps. Thus, the hybrid does not require a single global sparsity assumption: it selects the lower cost implementation for each attention map while preserving the same receiver model and context.

C.2 L2 Cache Optimization via Block Swizzling

A naive row-major execution of thread blocks can result in poor spatial locality, leading to frequent L2 cache misses when reading the receiver’s value cache and the linear transformation weights. To mitigate this, the kernel implements a thread block swizzling strategy. The kernel groups continuous row blocks into chunks defined by the `group_row` parameter. This forces thread blocks that are processing adjacent tokens and attention heads to execute concurrently. By localizing memory accesses, the swizzling technique significantly improves the L2 cache hit rate for both the receiver’s value cache and the broadcasted weight matrices. The optimal `group_row` size (typically 2, 4, or 8) is dynamically determined via Triton’s autotuning framework.

C.3 Split-K Reduction for SM Occupancy

In scenarios with small batch sizes or relatively short context lengths, the number of standard attention tiles may be insufficient to fully saturate the GPU’s Streaming Multiprocessors (SMs), leading to hardware underutilization. To address this, MAPFLOW utilizes a **Split-K reduction** approach.

The accumulation along the sequence dimension (the reduction axis for the dot product) is divided into `split_k` independent chunks. The kernel dynamically calculates an estimated `split_k` value based on the device’s SM count, the base tile count, and a target wave heuristic to ensure optimal occupancy. When Split-K is triggered, the operation is decoupled into a two-stage pipeline:

- (1) **Stage 1 (`_decompose_varlen_page_fused_splitk_stage1`):** Thread blocks compute the attention-value dot products for their assigned sub-segments of the sequence. The intermediate results are accumulated in FP32 and stored in a temporary global workspace (`partial_ptr`).
- (2) **Stage 2 (`_splitk_reduce_tiles`):** A secondary kernel reads the intermediate workspace, reduces the partial tiles across the `split_k` dimension, applies the final data type casting (e.g., FP16 or BF16), and writes the result to the final output buffer.

C.4 Register-Level Accumulation and Lazy Loading

The innermost loop of the kernel iterates over the non-zero blocks defined by the Block Sparse Row (BSR) metadata (compressed row indices, column indices, and values). To minimize global memory bandwidth consumption, the kernel applies a “lazy loading” flush mechanism.

As the kernel iterates through the sender’s sparse attention blocks, it performs the linear transformation ($A_{\text{block}} \times W$) and accumulates the mapped attention scores directly in fast, on-chip registers (`attn_acc`). The kernel strictly monitors the sequence position (`current_seq_pos`). The receiver’s value block is only fetched from global memory (via `_get_rcv_value_block`) when the sequence position boundary changes (`current_seq_pos` $\neq$ `last_seq_pos`). Once the boundary is crossed, a fused dot-product is executed against the accumulated attention scores, and the registers are zeroed out for the next block.

C.5 Variable-Length Sequence Packing and PagedAttention

To maintain compatibility with modern high-throughput serving architectures, the kernel natively supports continuous batching and paged memory management.

- **Variable-Length Packing:** The kernel packs multiple heterogeneous requests into a single continuous 1D array. It navigates these boundaries using cumulative offset pointers (`cu_crow_indices`, `cu_col_indices`, and `bsr_batch_offsets`).
- **Paged Memory:** The receiver’s value cache is entirely unflattened. The kernel accesses the physical memory pages dynamically through a `block_table` mapping, natively supporting the memory paradigms established by PagedAttention.

Received 17 April 2026; revised 19 August 2026; accepted 12 September 2026